RepoPilot -- Official Project Documentation

Open-Source Codebase Intelligence, Architecture & Evidence-Grounded Debugging Platform

OFFICIAL VERIFIED DEPLOYMENT & REPOSITORY METRICS

[Live Demo] Production Deployment: https://repo-pilot-git.vercel.app
[Source] GitHub Repository: https://github.com/Mohit-Rocky-25/RepoPilot
[Stack] Next.js 14 App Router, TypeScript (Strict), Three.js 3D, Tree-sitter WASM, Local Ollama ($0 API Spend)

1. What is RepoPilot?

RepoPilot is an independent, open-source codebase intelligence and developer productivity platform. It enables software engineers, first-time open-source contributors, and researchers to explore, understand, and debug unfamiliar repositories with precision.

Entering an unfamiliar software codebase is one of the most time-consuming activities in engineering. Developers spend a substantial portion of their daily working time reading existing code, deciphering architectural scaffolding, and locating the source of runtime errors before writing new logic. Most existing developer tools either rely on expensive cloud LLM subscriptions that can hallucinate non-existent files and symbols, or provide basic file trees without architectural context.

RepoPilot solves this through an evidence-first engineering approach:

- **Deterministic Core:** File navigation, AST symbol extraction, architecture detection, and ranked code search run entirely on local compute without requiring an AI model or cloud API.
- **Traceable Citations:** Every analysis, symbol jump, and assistant answer is tied to an immutable commit SHA, exact file path, and line coordinate.
- **Optional Private AI:** Supports local open-weights coding models via Ollama (e.g., qwen2.5-coder:7b) or user-supplied OpenAI-compatible API keys (sessionStorage only) with strict token bounds and automated citation verification. Operates 100% offline when AI is unavailable.
- **Defensive Sandboxing:** Untrusted repository code is never executed automatically. Bug fixes are proposed as unified diffs validated in memory against pinned file lines with reproducible regression tests for human review.

2. Key Capabilities & Implementation Status

Feature Area	Description	Status	No Key Demo?	Data Source
Repository Ingestion	Ingests public GitHub repos via streaming tarball, validates URLs against SSRF, pins commit SHA.	Stable	Yes	GitHub Tarball / REST API
Code Explorer	Resizable dual-pane tree and syntax-highlighted code viewer with line coordinates.	Stable	Yes	Extracted files (<100KB)
Deterministic Search	Multi-modal search ranking BM25 lexical matches, paths, entry points, and symbols.	Stable	Yes	In-memory search engine
AST Symbol Extraction	Real Tree-sitter WASM grammars (TS/JS/Py/Go/Rust) with heuristic regex fallback.	Beta	Yes	Tree-sitter WASM / regex
Architecture Map	Interactive 2D graph (default) with optional WebGL 3D orbital perspective map.	Beta	Yes	Manifest analysis & imports
Architecture Brief	Summarizes detected frameworks, confirmed application entry points, and directory roles.	Stable	Yes	Manifest inspection
Grounded Code AI	Conversational assistant with bounded context and strict citation verification against pinned SHA.	Beta	Yes (Offline)	Ollama / BYO Key / Offline
Bug & Stack Analyzer	Multi-language parser (Node/V8, Python chained, Go, Rust) with ANSI stripping and 64KB cap.	Beta	Yes	Stack frame parser & engine
Patch Review & PR	Unified diffs validated in memory against pinned files; "Unverified: review before applying".	Beta	Yes	In-memory diff validator
Rate Limit Mgmt	Platform-trusted IP derivation, Upstash Redis support with in-memory fallback, session PAT.	Stable	Yes	Platform headers & store

3. Representative Use Cases

- First-Time Open-Source Contributors: Quickly locate application entry points, configuration schemas, and test directories in unfamiliar repositories to understand where a bug fix or feature belongs.

- Engineers Joining a New Codebase: Survey dependencies, module hierarchies, and architecture briefs within seconds without waiting for bulky IDE background indexing tasks.

- Production Incident & Bug Triage: Paste an error log or stack trace from CI or production to isolate user-code frames, pinpoint root causes, and draft candidate minimal unified diffs.

- Privacy-Conscious Developers & Students: Explore and query codebases on local hardware without paying subscription fees or sending proprietary source code to hosted cloud APIs.

4. How RepoPilot Works

RepoPilot operates via a 6-stage evidence pipeline:

- 1. Ingest & Pin Revision: Validates repository URL against SSRF policy, downloads single gzipped tarball from codeload.github.com, and resolves default branch HEAD to an immutable commit SHA.

- 2. Filter & Index: Excludes binaries, build artifacts (node_modules/, dist/), and files over 100KB. Supported files are indexed for AST symbols via Tree-sitter WASM.

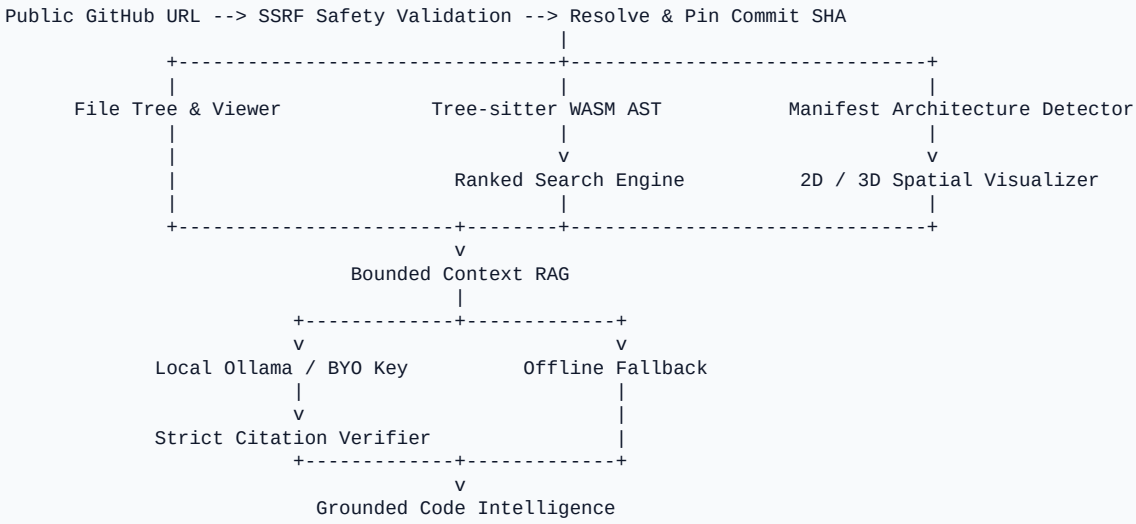
- 3. Map Architecture: Manifest files (package.json, Cargo.toml, pyproject.toml, go.mod) are parsed to detect frameworks and map module directory roles.

- 4. Search & Browse: Users navigate directory trees, search symbols with multi-factor BM25 ranking, and inspect code with line-level coordinates.

- 5. Contextual Q&A: Questions are matched against indexed source files, bounded to 4,000 tokens, and passed to local Ollama or BYO key. Citations returned are strictly verified against indexed files.

- 6. Investigate & Review: Runtime stack traces are parsed into structured frames, correlated with source files, and translated into ranked hypotheses and in-memory validated diffs for human review.

SYSTEM ARCHITECTURE DATA FLOW



5. Architecture & Design Decisions

A. Modular Monolith Architecture

RepoPilot packages adhere to strict Zod validation contracts:

- @repopilot/core: Shared schemas (RepositoryReference, SourceReference, BugHypothesis, PatchProposal).

- @repopilot/github: GitHub client with rate-limit tracking, tarball extraction, and commit revision pinning.

- @repopilot/indexer: Multi-language Tree-sitter WASM symbol extraction engine.

- **@repopilot/search:** Multi-factor BM25 and symbol ranking search engine.
- **@repopilot/analysis:** Framework detection and directory role heuristics.
- **@repopilot/ai:** ModelProvider abstraction (Ollama, BYO OpenAI-compatible, Offline) and citation verification.
- **@repopilot/debugging:** Multi-language stack trace parser and in-memory diff validator.
- **@repopilot/execution:** Execution safety policies and reproducible manual instructions.

B. Why Deterministic Tools Precede AI

AI models can hallucinate non-existent files, misquote syntax, or generate plausible-sounding errors. In RepoPilot, deterministic AST indexing, file tree navigation, and keyword search function completely independently of AI. AI is treated as an optional interpretive layer, never a prerequisite for basic navigation.

C. Human-in-the-Loop Patch Policy

RepoPilot adheres to strict developer safety standards:

- **Zero Remote Mutation:** It never applies patches automatically to local or remote repositories.
- **Zero Arbitrary Execution:** It never runs arbitrary build or test scripts on the server.
- **Human Verification:** All candidate diffs are presented in unified diff view for manual inspection and local reproduction labeled "Unverified: review before applying".

6. Honest Comparison Against Real Alternatives

Dimension	RepoPilot	DeepWiki	Sourcegraph	Cursor / Copilot
Setup	Zero setup (browser URL)	Web-based indexing	Enterprise self-host/cloud	IDE plugin config
Privacy	Local / sessionStorage keys	Cloud-hosted index	Enterprise on-prem/cloud	Cloud-hosted telemetry
Offline Core	Full offline core	No (hosted platform)	No (server required)	Partial (editor only)
Citations	Strict SHA verification	Document-level links	Compiler-grade SCIP graph	None (inline editor)
Cost	$0 / Free forever	Freemium / Subscription	Free tier / Enterprise $$$	$10-$20/mo per seat
Scope Limits	Public repos <= 25MB	Public & private repos	Multi-repo enterprise	Workspace indexing

Where RepoPilot is Weaker: No private repo OAuth; single-repo scope (no multi-repo cross-codebase linking); repository size bounds (25MB archive / 1,500 files); small local models (<7B) have less reasoning power than frontier cloud models; serverless rate limiting without Redis is best-effort.

7. Technology Stack

- **Framework:** Next.js 14 (App Router, React Server Components, Route Handlers)
- **Language:** TypeScript 5 (Strict Mode enabled with 0 errors)

- **UI & Layout:** React 18, Tailwind CSS tokens & Vanilla CSS components
- **Data Validation:** Zod 3 (Strict runtime contract validation across all API routes)
- **3D Visualization:** Three.js WebGL canvas and SVG-sanitized Mermaid architecture engine
- **Testing:** Vitest 2 (Unit, integration, and security suites, 95 tests passing across 14 files)
- **Local AI Daemon:** Ollama (Optional, supports qwen2.5-coder:7b, qwen2.5-coder:1.5b, etc.)

8. Quick Start & Setup Guide

GETTING STARTED LOCALLY

```
# 1. Clone the repository
git clone https://github.com/Mohit-Rocky-25/RepoPilot.git
cd RepoPilot

# 2. Install dependencies
npm install

# 3. Configure environment (Optional - safe defaults included out of the box)
cp .env.example .env.local

# 4. Start the development server
npm run dev

# Open http://localhost:3000 in your browser.
```

9. Development & Verification Commands

TESTING AND VERIFICATION SCRIPTS

```
# Run type checking (TypeScript Strict Mode)
npm run typecheck

# Run unit, integration, and security test suites (95 tests across 14 files)
npm test

# Build production bundle
npm run build

# Start production server
npm start
```

10. Security, Privacy & Known Limitations

RepoPilot was hardened following an exhaustive production application security audit:

- **SSRF & Cloud Metadata Defense:** Validates AI baseUrl and repo URLs. Strictly blocks 169.254.169.254, RFC1918 subnets, loopback, and IPv4-mapped IPv6.
- **Safe Tarball Ingestion:** Enforces 25MB archive cap, 100KB per-file cap, symlink root containment, and path traversal rejection.
- **Rate Limiting & Abuse Prevention:** Platform-trusted client IP extraction (x-vercel-forwarded-for), @upstash/ratelimit Redis support with in-memory fallback, 64KB body caps.
- **DOM-Based SVG XSS Defense:** Mermaid architecture SVGs sanitized via DOMParser, stripping <script>, inline event handlers, and javascript: URIs.
- **Prompt Injection Hardening:** Untrusted repository files isolated inside XML CDATA blocks (<untrusted_repo_content>).
- **Known Limitations:** Public repos only; 25MB archive / 1,500 files cap; Tree-sitter AST for TS/JS/Py/Go/Rust (heuristic for others); models <7B require human review; best-effort rate limiting without Redis.